# PLAN DE RECUPERACIÓN ANTE INCIDENTES

## Basado en NIST SP 800-61 — INC-2026-001

| Campo | Detalle |
|---|---|
| **Organización** | 4Geeks |
| **Analista responsable** | Gabriel Martinez |
| **Sistema afectado** | Servidor Debian 12 — IP 192.168.0.122 |
| **Referencia incidente** | INC-2026-001 |
| **Marco de referencia** | NIST SP 800-61 Rev. 2 |
| **Fecha** | Marzo 2026 |
| **Clasificación** | **CONFIDENCIAL** |

# 1. Introducción y Alcance

Este documento define el Plan de Recuperación ante Incidentes de Seguridad aplicado al incidente INC-2026-001 detectado en el servidor de producción de 4Geeks Academy. El plan sigue la metodología NIST SP 800-61 (Computer Security Incident Handling Guide) y documenta las 5 fases de respuesta ejecutadas.

## 1.1 Objetivo

- Documentar el proceso completo de detección, análisis, contención, erradicación y recuperación del incidente
- Establecer procedimientos reproducibles para futuros incidentes de seguridad
- Servir como base para el plan de mejora continua de la postura de seguridad

## 1.2 Sistema afectado

| Componente | Detalle |
|---|---|
| **Sistema operativo** | Debian GNU/Linux 12 (bookworm) — kernel 6.1.0-25-amd64 |
| **IP del servidor** | 192.168.0.122 (asignada por DHCP — confirmada por journalctl) |
| **Servicios activos** | Apache 2.4.62, MariaDB, OpenSSH 9.2p1, vsftpd 3.0.3 |
| **Aplicación web** | WordPress (instalado en /var/www/html/) |
| **Puertos expuestos** | 21/tcp (FTP), 22/tcp (SSH), 80/tcp (HTTP) — confirmado por nmap |
| **Puertos internos** | 3306/tcp (MariaDB — solo localhost), 631/tcp (CUPS — solo localhost) |

## 2. Roles y Responsabilidades

| Rol | Responsable | Responsabilidad |
|---|---|---|
| Analista Forense | Gabriel Martinez | Análisis, identificación y corrección de vulnerabilidades |
| Coordinador de Incidente | Instructor 4Geeks | Supervisión del proceso y validación de entregables |
| Administrador del sistema | Gabriel Martinez | Aplicación de correcciones en el servidor |
| Responsable de comunicaciones | 4Geeks Academy | Notificación a partes afectadas |

# 3. Fases de Respuesta al Incidente (NIST SP 800-61)

## Fase 1 — Preparación

La fase de preparación establece la capacidad de respuesta antes de que ocurra un incidente. En este caso, el entorno fue configurado intencionalmente con vulnerabilidades para el ejercicio, y se dispuso de las herramientas necesarias para el análisis.

### Herramientas utilizadas:

- Kali Linux — plataforma de análisis externo con nmap 7.95
- Debian 12 — sistema objetivo con acceso a bash, ss, grep, journalctl
- MariaDB — cliente de base de datos para verificación de credenciales
- Archivos de configuración: /etc/vsftpd.conf, /etc/ssh/sshd_config, /etc/apache2/apache2.conf

## Fase 2 — Detección e Identificación

La detección se realizó mediante análisis activo del sistema. No existían alertas automáticas previas dado que /var/log/auth.log no existía y los logs de Apache y vsftpd estaban vacíos.

### Cronología de la detección:

| Timestamp | Evento |
|---|---|
| 03/03 14:57 | Inicio del análisis — sistema activo 11 min, usuario debian en tty7 |
| 03/03 15:09 | Confirmado: /var/log/auth.log inexistente. Logs de Apache y vsftpd vacíos |
| 03/03 15:25 | Revisión de bash_history — se reconstruye proceso de instalación insegura |
| 03/03 18:45 | Análisis de logs Apache y vsftpd confirmados vacíos. Journalctl revisado. |
| 03/03 18:59 | IP confirmada: 192.168.0.122 (journalctl muestra asignación DHCP el Mar 04 00:08) |
| 03/03 19:25 | Escaneo nmap desde Kali Linux: puertos 21, 22, 80 abiertos externamente confirmados |
| 03/03 19:54 | Análisis de vsftpd.conf, sshd_config y apache2.conf completado con evidencia real |

### Indicadores de compromiso (IoC) identificados:

- wp-config.php con permisos -rwxrwxrwx (777) y credencial DB_PASSWORD='123456'
- AUTH_KEY y NONCE_KEY de WordPress sin cambiar ('put your unique phrase here')
- vsftpd con anonymous_enable=YES, write_enable=YES y ssl_enable=NO
- sshd con PermitRootLogin yes y PasswordAuthentication yes activos
- Options Indexes activo en apache2.conf en líneas 160, 167 y 172

- Servicios CUPS y Avahi activos sin función en servidor web
- /var/log/auth.log inexistente — imposible auditar accesos SSH históricos

## Fase 3 — Contención

La contención tuvo como objetivo evitar que las vulnerabilidades fueran explotadas mientras se preparaba la corrección definitiva.

### Acciones de contención inmediata:

1. Detención del servicio FTP: sudo systemctl stop vsftpd
2. Detención de CUPS y Avahi: sudo systemctl stop cups avahi-daemon
3. Verificación de que MariaDB (3306) solo escucha en localhost — no requirió acción
4. Revisión de que no existían usuarios no autorizados con shell activa
5. Revisión de cron jobs — sin tareas maliciosas detectadas

```
# Verificación de que MySQL NO está expuesto externamente
$ sudo grep -E 'bind-address' /etc/mysql/mariadb.conf.d/50-server.cnf
bind-address = 127.0.0.1   # correcto — MySQL solo accesible localmente

# Verificación de usuarios del sistema
$ awk -F: '($3 == 0) {print}' /etc/passwd
root:x:0:0:root:/root:/bin/bash   # solo root tiene UID 0 — sin backdoors
```

## Fase 4 — Erradicación

La erradicación eliminó las configuraciones inseguras y aplicó correcciones permanentes a cada vulnerabilidad identificada.

| ID | Vulnerabilidad | Corrección aplicada | Estado |
|---|---|---|---|
| V-01 | MySQL — contraseñas débiles | ALTER USER con nuevas contraseñas fuertes. wp-config.php actualizado. | ✓ CORREGIDO |
| V-02 | FTP — acceso anónimo sin SSL | anonymous_enable=NO, ssl_enable=YES, write_enable=NO. Servicio deshabilitado. | ✓ CORREGIDO |
| V-03 | SSH — PermitRootLogin yes | PermitRootLogin no, PasswordAuthentication no, MaxAuthTries 3. | ✓ CORREGIDO |
| V-04 | wp-config.php — permisos 777 | chmod 640, chown www-data:www-data. | ✓ CORREGIDO |
| V-05 | Apache — Options Indexes | Options -Indexes +FollowSymLinks en líneas 160, 167 y 172 de apache2.conf. | ✓ CORREGIDO |
| V-06 | CUPS y Avahi innecesarios | systemctl disable cups avahi-daemon. ufw deny 631/tcp y 5353/udp. | ✓ CORREGIDO |

## Fase 5 — Recuperación

La recuperación restauró todos los servicios en estado seguro y verificó el funcionamiento correcto del sistema.

### Verificaciones post-corrección:

```
# Verificar SSH con nueva configuración
$ sudo grep -E 'PermitRootLogin|PasswordAuthentication|MaxAuthTries'
/etc/ssh/sshd_config
PermitRootLogin no          # corregido
PasswordAuthentication no   # corregido
MaxAuthTries 3              # corregido

# Verificar Apache sin Indexes
$ sudo grep -n 'Options' /etc/apache2/apache2.conf | grep -v '#'
160: Options -Indexes +FollowSymLinks   # corregido
167: Options -Indexes +FollowSymLinks   # corregido
172: Options -Indexes +FollowSymLinks   # corregido

# Verificar permisos wp-config.php
$ ls -la /var/www/html/wp-config.php
-rw-r----- 1 www-data www-data 3017 wp-config.php   # corregido (antes 777)

# Verificar FTP deshabilitado
$ sudo systemctl is-enabled vsftpd
disabled   # corregido
```

## 4. RTO y RPO por Servicio

El RTO (Recovery Time Objective) define el tiempo máximo aceptable para restaurar un servicio. El RPO (Recovery Point Objective) define la pérdida máxima de datos aceptable.

| Servicio | RTO | RPO | Notas |
|---|---|---|---|
| WordPress / Apache (HTTP) | 1 hora | 24 horas | Servicio crítico — restaurar primero |
| MariaDB | 2 horas | 1 hora | Requiere backup previo a correcciones |
| SSH | 30 min | N/A | Acceso de administración — prioridad alta |
| FTP (vsftpd) | 4 horas | N/A | Servicio deshabilitado — no crítico |
| CUPS / Avahi | N/A | N/A | Servicios deshabilitados permanentemente |

# 5. Plan de Comunicación

## 5.1 Notificaciones durante el incidente

| Audiencia | Canal | Frecuencia | Responsable |
|---|---|---|---|
| Instructor / Coordinador | Email / Presencial | Al inicio y cierre | Gabriel Martinez |
| Dirección 4Geeks | Informe escrito | Al cierre del incidente | Coordinador |
| Equipo de TI | Documentos técnicos | Tras cada fase | Gabriel Martinez |

## 5.2 Documentos generados

- Informe de Pentesting — análisis técnico completo con evidencia real
- Informe de Incidente de Seguridad (INC-2026-001) — cronología y correcciones
- Plan de Recuperación ante Incidentes (este documento) — basado en NIST SP 800-61
- Presentación ejecutiva — resumen para audiencia no técnica

## 6. Política de Backups — Regla 3-2-1

Se recomienda implementar la regla 3-2-1 para todos los datos críticos del servidor:

| Regla | Principio | Aplicación en este servidor |
|---|---|---|
| **3** | 3 copias de los datos | Datos en producción + backup local en /backup + backup remoto |
| **2** | 2 medios diferentes | Disco local del servidor + almacenamiento externo (NAS o nube) |
| **1** | 1 copia offsite | Backup en ubicación física diferente o servicio cloud (S3, Backblaze) |

### Componentes críticos a respaldar:

- /var/www/html/ — archivos de WordPress
- /var/www/html/wp-config.php — configuración con credenciales (cifrada)
- Base de datos: mysqldump wordpress > /backup/wordpress_$(date +%Y%m%d).sql
- /etc/ssh/sshd_config, /etc/vsftpd.conf, /etc/apache2/ — configuraciones de servicios
- /etc/mysql/ — configuración de MariaDB

# 7. Plan de Mejora Continua

## 7.1 Acciones inmediatas (esta semana)

- Implementar autenticación SSH por clave pública — eliminar acceso por contraseña
- Instalar y configurar fail2ban para SSH y FTP
- Activar HTTPS con certificado Let's Encrypt (puerto 443)
- Generar claves AUTH_KEY y NONCE_KEY válidas en wp-config.php
- Configurar rsyslog para generar /var/log/auth.log correctamente

## 7.2 Corto plazo (este mes)

- Implementar WAF (ModSecurity) para WordPress
- Configurar AIDE para monitoreo de integridad de archivos
- Ejecutar escaneos automáticos con Lynis o OpenVAS
- Deshabilitar X11Forwarding en sshd_config (actualmente YES — innecesario)
- Establecer política formal de contraseñas con gestor centralizado

## 7.3 Mediano plazo (3 meses)

- Implementar SIEM para centralización y análisis de logs
- Realizar auditoría de seguridad externa con empresa certificada
- Capacitar al equipo de TI en checklist de hardening para nuevos servidores
- Evaluar implementación de ISO 27001

## 7.4 Checklist de seguridad para futuras instalaciones

| Verificación | OK | N/A |
|---|---|---|
| SSH: PermitRootLogin no | [ ] | [ ] |
| SSH: PasswordAuthentication no (usar claves) | [ ] | [ ] |
| SSH: MaxAuthTries configurado (máx. 3) | [ ] | [ ] |
| FTP: anonymous_enable=NO o servicio deshabilitado | [ ] | [ ] |
| Apache: Options -Indexes configurado | [ ] | [ ] |
| WordPress: wp-config.php con chmod 640 | [ ] | [ ] |
| WordPress: AUTH_KEY y NONCE_KEY generadas correctamente | [ ] | [ ] |
| MySQL: contraseñas fuertes y bind-address=127.0.0.1 | [ ] | [ ] |
| Firewall: ufw activo con reglas mínimas necesarias | [ ] | [ ] |
| Logs: /var/log/auth.log activo y accesible | [ ] | [ ] |

| | | |
|---|---|---|
| Servicios innecesarios (CUPS, Avahi) deshabilitados | [ ] | [ ] |

## 8. Conclusión

El incidente INC-2026-001 fue contenido, erradicado y recuperado en menos de 24 horas desde el inicio del análisis. Las 6 vulnerabilidades identificadas fueron corregidas en su totalidad, y el servidor Debian 12 opera actualmente con una configuración segura.

El análisis forense determinó que las vulnerabilidades fueron introducidas durante la configuración inicial del servidor, no como resultado de un ataque externo activo. La ausencia de /var/log/auth.log imposibilitó verificar el historial completo de accesos SSH, lo cual debe corregirse como medida preventiva prioritaria.

Este plan de recuperación, junto con el informe de pentesting y el informe de incidente, constituyen la documentación completa del ejercicio de seguridad de la Fase 1 del proyecto final de 4Geeks Academy.

| | |
|---|---|
| **Gabriel Martinez**<br>Analista de Ciberseguridad<br>4Geeks— Marzo 2026 | **Referencia: NIST SP 800-61 Rev. 2**<br>INC-2026-001 — Estado: CERRADO |